

Python Assessment 3

1.

```
Python Assessment 3

1. Create Arrays Create the following arrays: • Using a Python list • Using np.array(), • Using np.arange(), • Using np.linspace()

[3]
✓ 0s
import numpy as np
arr = [10,15,20,25,30]
arr1 = np.array([10,20,30,40,50])
arr2=np.arange(1,13)
arr3=np.linspace(1,10,5)
print(arr)
print(arr1)
print(arr2)
print(arr3)

[10, 15, 20, 25, 30]
[10 20 30 40 50]
[ 1  2  3  4  5  6  7  8  9 10 11 12]
[ 1.  3.25  5.5  7.75 10. ]
```

2.

```
2. Array Properties Create an array of numbers from 1 to 30 and print: • Shape, • Size, • Data type, • Number of dimensions, • Item size,
• Total memory used

[10]
✓ 0s
import numpy as np
arr =np.arange(1,31)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.ndim)
print(arr.itemsize)
print(arr.nbytes)

(30,)
30
int64
1
8
240
```

3.

```
3. Array Creation Functions Create arrays using: • zeros(), • ones(), • empty(), • full(), • eye(), • identity(), Explain the difference between
each.

[12]
0s
import numpy as np
arr = np.zeros((5,5))
arr1=np.ones((5,5))
arr2=np.full((3,3),77)
arr3=np.eye(5)
arr4=np.identity(4)
arr5=np.empty((3,3))
print(arr)
print("creates an array filled with 0")
print(arr1)
print("creates an array filled with 1")
print(arr2)
print("creates an array where every element is the same value")
print(arr3)
print("creates an identity matrix with 1's on the main diagonal")
print(arr4)
print("same creates an identity matrix")
print(arr5)
print("it creates an array without initializing values and show whatever in the memory location")
```

```

... [[0. 0. 0. 0. 0.]
      [0. 0. 0. 0. 0.]
      [0. 0. 0. 0. 0.]
      [0. 0. 0. 0. 0.]
      [0. 0. 0. 0. 0.]]
creates an array filled with 0
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
creates an array filled with 1
[[77 77 77]
 [77 77 77]
 [77 77 77]]
creates an array where every element is the same value
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
creates an identity matrix with 1's on the main diagonal
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
same creates an identity matrix
[[3.8e-322 3.8e-322 3.8e-322]
 [3.8e-322 3.8e-322 3.8e-322]
 [3.8e-322 3.8e-322 3.8e-322]]
it creates an array without initializing values and show whatever in the memory location

```

4.

4. Reshaping Arrays Create an array from 1-24. Convert it into: • 2x12, • 3x8, • 4x6, • 6x4, • 8x3, • 12x2

```

144
0s
import numpy as np
arr = np.arange(1,25)
print(arr.reshape(2,12))
print(arr.reshape(3,8))
print(arr.reshape(4,6))
print(arr.reshape(6,4))
print(arr.reshape(8,3))
print(arr.reshape(12,2))

[[ 1  2  3  4  5  6  7  8  9 10 11 12]
 [13 14 15 16 17 18 19 20 21 22 23 24]]
[[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]
 [17 18 19 20 21 22 23 24]]
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

```

ables Terminal 1:51 PM Python 3

```

[17 18 19 20 21 22 23 24]]
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]
 [16 17 18]
 [19 20 21]
 [22 23 24]]
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]
 [13 14]
 [15 16]
 [17 18]
 [19 20]
 [21 22]
 [23 24]]

```

ables Terminal 1:51 PM Python 3

5.

5. Flattening Arrays Create a 4x4 matrix. Convert it into a 1D array using: • `flatten()`, • `ravel()`, Explain the difference.

```
[28] import numpy as np
✓ Os arr = np.array([[1,2,3,4],
                    [5,6,7,8],
                    [9,10,11,12],
                    [13,14,15,16]])

print(arr.shape)
print(arr.flatten())
print(arr.ravel())
print("Both convert the nxm matrix into 1d matrix")
```

▼

```
(4, 4)
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16]
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16]
Both convert the nxm matrix into 1d matrix
```

6.

6. Indexing and Slicing Given `arr = np.array([5,10,15,20,25,30,35,40,45,50])` Print: • First element, • Last element, • Middle element, • First five values, • Last four values, • Alternate elements, • Reverse the array

```
[8] import numpy as np
✓ Os arr = np.array([5,10,15,20,25,30,35,40,45,50])

print(arr[0])
print(arr[-1])
print(arr[len(arr)//2])
print(arr[:5])
print(arr[:5:-1])
print(arr[::-2])
print(arr[::-1])
```

▼

```
... 5
50
30
[ 5 10 15 20 25]
[50 45 40 35]
[ 5 15 25 35 45]
[50 45 40 35 30 25 20 15 10 5]
```

7.

7. Fancy Indexing Given `arr = np.array([10,20,30,40,50,60,70,80])` Print elements at index: • 1, • 3, • 5, • 7, using Fancy Indexing

```
[48] import numpy as np
✓ Os arr = np.array([10,20,30,40,50,60,70,80])

print(arr[[1,3,5,7]])
```

▼

```
... [20 40 60 80]
```

8.

8. Boolean Indexing Given `marks = np.array([45,78,82,56,91,39,67,88])` Display: • Marks greater than 75, • Marks less than 50, • Marks between 60 and 90

```
[45] import numpy as np
✓ Os marks = np.array([45,78,82,56,91,39,67,88])

print(marks[marks > 75])
print(marks[marks < 50])
print(marks[(marks > 60) & (marks < 90)])
```

▼

```
... [78 82 91 88]
[45 39]
[78 82 67 88]
```

9.

9. Arithmetic Operations Create two arrays. Perform: • Addition • Subtraction • Multiplication • Division • Floor division • Modulus • Exponentiation

```
import numpy as np
n = np.array([1,2,3,4,5])
m = np.array([6,7,8,9,10])
print(n+m)
print(n-m)
print(n*m)
print(n/m)
print(n//m)
print(n%m)
```

```
... [ 7  9 11 13 15]
     [-5 -5 -5 -5 -5]
     [ 6 14 24 36 50]
     [0.16666667 0.28571429 0.375      0.44444444 0.5      ]
     [1 2 3 4 5]
     [ 1      128    6561   262144  9765625]
```

10.

10. Broadcasting Create: • One array of shape (3,3), • Another array of shape (3,), Perform broadcasting and explain how NumPy applies it.

```
import numpy as np
arr = np.array([[1,2,3],
               [5,6,7]])
arr1 = np.array([1,2,3])
print(arr + arr1)
```

```
[[ 2  4  6]
 [ 6  8 10]]
```

11.

11. Universal Functions (ufuncs) Using one NumPy array, perform: • sqrt() • square() • power() • abs() • sin() • cos() • tan() • exp() • log()

```
import numpy as np
arr = np.array([20,45,32,66,12,43])
print(np.sqrt(arr))
print(np.square(arr))
print(np.power(arr,2))
print(np.abs(arr))
print(np.sin(arr))
print(np.cos(arr))
print(np.tan(arr))
print(np.exp(arr))
print(np.log(arr))
```

```
... [4.47213595  6.70820393  5.65685425  8.1240384   3.46410162  6.55743852]
     [ 400 2025 1024 4356  144 1849]
     [ 400 2025 1024 4356  144 1849]
     [20 45 32 66 12 43]
     [ 0.91294525  0.85090352  0.55142668 -0.02655115 -0.53657292 -0.83177474]
     [ 0.40808206  0.52532199  0.83422336 -0.99964746  0.84385396  0.5551133 ]
     [ 2.23716094  1.61977519  0.66100604  0.02656052 -0.63585993 -1.49838734]
     [4.85165195e+08  3.49342711e+19  7.89629602e+13  4.60718663e+28
     1.62754791e+05  4.72783947e+18]
     [2.99573227  3.80666249  3.4657359   4.18965474  2.48490665  3.76120012]
```

12.

12. Statistical Functions Given salary = np.array([25000,32000,45000,50000,60000,28000,70000]) Find: • Maximum • Minimum • Mean • Median • Standard deviation • Variance • Sum • Product

```
[129]
✓ Os
import numpy as np
salary = np.array([25000,32000,45000,50000,60000,28000,70000])
print(salary.max())
print(salary.min())
print(salary.mean())
print(np.median(salary))
print(salary.std())
print(salary.sum())
print(np.var(salary))
print(np.prod(salary))
```

```
✓
... 70000
25000
44285.71428571428
45000.0
15718.181335313444
310000
247061224.48979592
-7767379107058286592
```

13.

13. Sorting Arrays Create an unsorted array. Perform: • Ascending sort • Descending sort • Find index after sorting using argsort()

```
[69]
✓ Os
import numpy as np
arr = np.array([2,5,4,7,8,1,3,6])
print(np.sort(arr))
print(np.sort(arr)[::-1])
print(np.argsort(arr))
```

```
✓
[1 2 3 4 5 6 7 8]
[8 7 6 5 4 3 2 1]
[5 0 6 2 1 7 3 4]
```

14.

14. Searching Elements Given arr = np.array([5,10,15,20,25,30,35,40]) Find: • Position of 25 • Position of 40 • Whether 50 exists or not

```
[72]
✓ Os
import numpy as np
arr = np.array([5,10,15,20,25,30,35,40])
print(np.where(arr == 25)[0][0])
print(np.where(arr == 40)[0][0])
if 50 in arr:
    print("it exists")
else:
    print("it doesn't exist")
```

```
✓
... 4
7
it doesn't exist
```

15.

15. Unique Values Given arr = np.array([2,3,5,3,2,6,8,5,6,9]) Find: • Unique values • Count of each unique value

```
[76]
import numpy as np
arr = np.array([2,3,5,3,2,6,8,5,6,9])
print(np.unique(arr))
unique,counts = np.unique(arr,return_counts=True)
print(counts)
```

```
✓
[2 3 5 6 8 9]
[2 2 2 2 1 1]
```

16.

16. Joining Arrays Create two arrays. Join them using: • concatenate() • stack() • vstack() • hstack() • dstack()

```
[81]
✓ Os
import numpy as np
arr=[1,2,3,4,5]
arr1=[6,7,8,9,10]
print(np.concatenate((arr,arr1)))
print(np.stack((arr,arr1)))
print(np.vstack((arr,arr1)))
print(np.hstack((arr,arr1)))
print(np.dstack((arr,arr1)))
```

▼

```
[ 1  2  3  4  5  6  7  8  9 10]
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
[ 1  2  3  4  5  6  7  8  9 10]
[[[ 1  6]
   [ 2  7]
   [ 3  8]
   [ 4  9]
   [ 5 10]]]
```

17.

17. Splitting Arrays Create an array from 1–24. Split it using: • split() • hsplit() • vsplit() Explain the output.

```
[98]
✓ Os
import numpy as np
arr =np.arange(1,25)
print(np.split(arr,4))
arr1 = arr.reshape(4,6)
print(np.hsplit(arr1,3))
print(np.vsplit(arr1,2))
```

▼

```
... [array([1, 2, 3, 4, 5, 6]), array([ 7,  8,  9, 10, 11, 12]), array([13, 14, 15, 16, 17, 18]), array([19, 20, 21, 22, 23, 24])]
[array([[ 1,  2],
       [ 7,  8],
       [13, 14],
       [19, 20]]), array([[ 3,  4],
       [ 9, 10],
       [15, 16],
       [21, 22]]), array([[ 5,  6],
       [11, 12],
       [17, 18],
       [23, 24]])]
[array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]]), array([[13, 14, 15, 16, 17, 18],
       [19, 20, 21, 22, 23, 24]])]
```

18.

18. Copy vs View Create an array. Create: • A copy using copy() • A view using view() Modify the original array and explain what happens in both cases

```
[84]
✓ Os
import numpy as np
arr =np.array([1,2,3,4,5,6,7])
arr2=np.copy(arr)
print(arr2)
view=arr.view()
arr[0]=40
print(view)
```

▼

```
[1 2 3 4 5 6 7]
[40 2 3 4 5 6 7]
```

19.

19. Random Module Generate: • Random integers • Random floats • Random matrix • Identity matrix with random values • Shuffle the array • Generate random numbers using a seed

```
[195] import numpy as np
✓ Os arr = np.array([1,2,3,4,5,6,7,8,9,10])
print(np.random.randint(1,20,5))
print(np.random.rand(5))
print(np.random.rand(3,3))
iden = np.eye(3)
ran= np.random.rand(3, 3)
print(iden * ran)
np.random.shuffle(arr)
print(arr)
np.random.seed(10)
print(np.random.randint(1,100,5))

... [10  5 16  1 18]
[0.74880388 0.49850701 0.22479665 0.19806286 0.76053071]
[[0.16911884 0.08833981 0.68535982]
 [0.95339335 0.00394827 0.51219226]
 [0.81262096 0.61252607 0.72175532]]
[[0.29187607 0.         0.        ]
 [0.         0.14217005 0.        ]
 [0.         0.         0.43401399]]
[ 2  7  9  8  1  6  3  4  5 10]
[10 16 65 29 90]
```

20. Mini Project

20. Mini Project – Student Marks Analysis

A class has marks of 50 students stored in a NumPy array.

Perform the following analysis:

- Find highest mark
- Find lowest mark
- Find average mark
- Find median mark
- Find standard deviation
- Count students who scored above 75
- Count students who failed (marks < 40)
- Display top 5 highest marks
- Display bottom 5 lowest marks
- Sort marks in ascending and descending order
- Replace all failed marks with 40 using Boolean Indexing
- Display the updated marks array

```
[126] import numpy as np
✓ Os marks = np.random.randint(0,101,50)
print(marks)
print("highest mark")
print(np.max(marks))
print("lowest mark")
print(np.min(marks))
print("average mark")
print(np.average(marks))
print("median")
print(np.median(marks))
print("std")
print(np.std(marks))
print("Count students who scored above 75")
print(np.sum(marks>75))
print("Count students who failed (marks < 40)")
print(np.sum(marks<40))
print(" Display top 5 highest marks")
print(np.sort(marks)[-6:-1])
print("Display bottom 5 lowest marks")
print(np.sort(marks)[:5])
print("sort marks in ascending and descending order")
print("ascending")
print(np.sort(marks))
print("descending")
print(np.sort(marks)[::-1])
print("Replace all failed marks with 40 using Boolean Indexing")
marks[marks < 40] = 40
print(marks)
```

Variables Terminal

✓ 1:59 PM Python 3

```

[52 88 91 42 68 4 86 51 30 53 31 78 71 1 14 67 71 45 59 50 54 37 3 65
20 10 62 99 54 19 2 19 95 69 73 99 95 1 93 14 93 14 96 8 88 62 4 84
9 75]
highest mark
99
lowest mark
1
average mark
51.26
median
54.0
std
32.3572310311003
Count students who scored above 75
13
Count students who failed (marks < 40)
18
Display top 5 highest marks
[99 99 96 95 95]
Display bottom 5 lowest marks
[1 1 2 3 4]
Sort marks in ascending and descending order
ascending
[ 1 1 1 2 3 4 4 8 9 10 14 14 14 19 19 20 30 31 37 42 45 50 51 52 53
54 54 59 62 62 65 67 68 69 71 71 73 75 78 84 86 88 88 91 93 93 95 95 96
99 99]
descending
[99 99 96 95 95 93 93 91 88 88 86 84 78 75 73 71 71 69 68 67 65 62 62 59
54 54 53 52 51 50 45 42 37 31 30 20 19 19 14 14 14 10 9 8 4 4 3 2
1 1]
Replace all failed marks with 40 using Boolean Indexing
[52 88 91 42 68 40 86 51 40 53 40 78 71 40 40 67 71 45 59 50 54 40 40 65
40 40 62 99 54 40 40 40 95 69 73 99 95 40 93 40 93 40 96 40 88 62 40 84
40 75]

```

Variables Terminal ✓ 1:59 PM Python 3

```

Count students who failed (marks < 40)
18
Display top 5 highest marks
[99 99 96 95 95]
Display bottom 5 lowest marks
[1 1 2 3 4]
Sort marks in ascending and descending order
ascending
[ 1 1 1 2 3 4 4 8 9 10 14 14 14 19 19 20 30 31 37 42 45 50 51 52 53
54 54 59 62 62 65 67 68 69 71 71 73 75 78 84 86 88 88 91 93 93 95 95 96
99 99]
descending
[99 99 96 95 95 93 93 91 88 88 86 84 78 75 73 71 71 69 68 67 65 62 62 59
54 54 53 52 51 50 45 42 37 31 30 20 19 19 14 14 14 10 9 8 4 4 3 2
1 1]
Replace all failed marks with 40 using Boolean Indexing
[52 88 91 42 68 40 86 51 40 53 40 78 71 40 40 67 71 45 59 50 54 40 40 65
40 40 62 99 54 40 40 40 95 69 73 99 95 40 93 40 93 40 96 40 88 62 40 84
40 75]
Display the updated marks array
[52 88 91 42 68 40 86 51 40 53 40 78 71 40 40 67 71 45 59 50 54 40 40 65
40 40 62 99 54 40 40 40 95 69 73 99 95 40 93 40 93 40 96 40 88 62 40 84
40 75]

```